VERIQTA

20 PAGES

Your DevOps Resume:

The 2026 Interview Playbook

Build a resume, show real skills, and get ready for DevOps interviews with confidence.





- Resume Templates
- Project Examples
- Interview Questions
- Real Scenarios
- Practical Tips



RESUME GUIDE | PROJECT STRATEGY | INTERVIEW PRACTICE | REAL EXAMPLES | JOB READINESS

Follow @veriqta

@veriqta | @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA

The 2026 DevOps Hiring Reality

Why This Matters

Not all "junior" DevOps roles are the same. Some expect basic knowledge, while others expect real hands-on experience. You need to understand what employers are actually looking for so you can build the right skills, projects, and resume.

What You Will Learn

- ✅ What junior DevOps, cloud, platform, and SRE roles really expect in 2026
- ✅ How to separate essential skills from nice-to-have skills
- ✅ How to spot unrealistic requirements in "junior" job listings
- ✅ How to extract skills, evidence, and gaps from a real job description

Common Role Titles

- Junior DevOps Engineer
- Junior Cloud Engineer
- Junior Platform Engineer
- Junior Site Reliability Engineer (SRE)
- Associate DevOps Engineer
- DevOps Intern

Core Areas Employers Look For

Linux	Git	Bash Scripting	Docker	Kubernetes
Cloud (AWS/Azure/GCP)	CI/CD	Infrastructure as Code (Terraform)	Monitoring & Logging	Security Basics

Example Job Description (Junior DevOps Engineer)

We are looking for a **Junior DevOps Engineer** to support our infrastructure and deployment processes. You will work with **Linux, AWS, Docker, Kubernetes, and CI/CD tools.** You should have experience with **scripting, version control (Git),** and **monitoring tools.** Familiarity with **Terraform** is a plus. You will help automate deployments, troubleshoot issues, and work with the team to improve system reliability.

Key Observation

- This is a "junior" role, but it expects mulitple tools.
- Look for what is required vs what is optional.
- Identify where you have evidence and where you need more practice.

Break It Down: Skills, Evidence and Gaps

Skill / Technology	Type	Evidence You Need	Your Gap?
Linux	Required	Commands, file permissions, services, logs	✅
AWS	Required	Real project or labs (e.g. EC2, IAM, S3, VPC)	❌
Docker	Required	Containerize an app, use volumes, Dockerfile	❌
Kubernetes	Required	Deploy an app, work with Pods, Services, troubleshooting	❌
CI/CD	Required	Build a pipeline (GitHub Actions, GitLab CI, or Jenkins)	❌
Terraform	Optional (Plus)	Provision simple infrastructure (e.g. VPC, EC2, S3)	❌
Monitoring	Required	Use tools like Prometheus, Grafana or CloudWatch	❌

Practical Focus

1. Read a real job description.
2. Highlight required vs optional skills.
3. List the evidence you already have.
4. Identify your gaps.
5. Plan what to learn or build next.

Remember This

- Job titles can be misleading.
- "Junior" does not always mean beginners.
- Focus on practical experience, not just tools.
- Show real evidence in your resume and GitHub.
- Build and document projects that match what employers are looking for.

VERIQTA

Your Resume Has Two Readers

Learn how applicant tracking systems (ATS) and human reviewers interact with your resume. Understand parsing, searchable skills, readable structure, and why keyword stuffing cannot replace evidence.

1. Applicant Tracking System (ATS)



- Parses your resume into sections
- Finds and matches keywords
- Ranks your resume based on relevance
- May filter out weak or unstructured resumes

2. Human Reviewer (Recruiter / Hiring Manager)



- Checks if your experience matches the role
- Looks for clear, readable structure
- Validates technical skills and real evidence
- Assesses projects, achievements, and impact
- Decides if you move to the next stage

How ATS Parses Your Resume



If your resume has:

- ✗ Unusual formatting
- ✗ Graphics or columns
- ✗ Missing section headers
- ✗ Skills hidden in paragraphs
- ✗ PDF with complex design

ATS may miss important information.

Why Keyword Stuffing Does Not Work

- ✗ Listing too many tools without context
- ✗ Copying keywords from the job description
- ✗ Skills that you cannot explain in an interview

What Works Instead

- ✓ Relevant skills
- ✓ Real projects
- ✓ Clear experience
- ✓ Evidence you can discuss and demonstrate

Poorly Structured Resume (Example)

JOHN DOE
johndoe123@gmail.com | +123456789

SUMMARY
Hardworking and passionate DevOps engineer with experience in many tools and technologies.
→ No clear structure

SKILLS
Linux, AWS, Docker, Kubernetes, Terraform, Jenkins, Git, CI/CD, Ansible, Python, Monitoring, Networking, Security, Cloud, DevOps, Scripting, Shell, Databases, Grafana, Prometheus, GitHub, Agile, Automation, Web, Nginx, MySQL, EC2, S3, VPC, IAM, Azure, GCP
→ Skills list is too long and not organized

EXPERIENCE
Worked on various DevOps tasks and projects.
→ No real achievements or details

PROJECTS
Personal project using AWS and Docker.
→ Projects are not explained

EDUCATION
BSc Computer Science.
→ Missing important information

Clean and Readable Resume (Example)

JOHN DOE
Junior DevOps Engineer
johndoe@gmail.com | +123456789 | github.com/johndoe
linkedin.com/in/johndoe
→ Clear and professional structure

PROFESSIONAL SUMMARY
Junior DevOps Engineer with hands-on experience in Linux, AWS, Docker, Kubernetes, and CI/CD. Built and deployed real projects, automated infrastructure, and troubleshoot system issues.
→ Relevant and concise summary

KEY SKILLS
Linux | AWS | Docker | Kubernetes | CI/CD (GitHub Actions) Terraform | Monitoring (Prometheus, Grafana) | Bash | Git
→ Well-organized skills section

PROJECTS
- Deployed a containerized web application on AWS using Docker and Terraform. [GitHub link]
- Built a CI/CD pipeline with GitHub Actions for automated testing and deployment. [GitHub link]

→ Real project examples with links

EDUCATION
BSc Computer Science, 2020 – 2024
→ Clean and easy to read

Practical Focus: Compare and Improve

1. Review the poorly structured resume and identify the problems.
2. Review the clean resume and note what makes it effective.
3. Rewrite one of your own resume sections using the clean format.

Key Takeaway

Your resume must be both ATS-friendly and human-friendly. List relevant skills, show real evidence, and keep it clear and readable.

VERIQTA

The Anatomy of a Credible DevOps Resume

Build the correct hierarchy for contact information, professional summary, technical skills, projects, work experience, education, and certifications.

1 Contact Information

- Your full name
- Professional title (e.g. Junior DevOps Engineer)
- Email address
- Phone number
- Location (City, Country)
- Links: GitHub, LinkedIn, Portfolio (optional)

2 Professional Summary

- 2–4 lines about who you are
- Key skills and strengths
- Target role
- Be specific and honest
- Do not exaggerate

3 Technical Skills

- Group by category (e.g. Cloud, DevOps Tools, OS)
- List the tools you have actually used
- Keep it clear and organized

4 Projects

- Real hands-on projects
- Show what you built and why
- Mention key tools used
- Add a link to your GitHub repo
- Highlight what you learned

5 Work Experience

- Paid, internship, or relevant experience
- Use clear bullet points
- Focus on actions and impact
- If no DevOps job yet, include relevant experience or projects

6 Education

- Degree or diploma
- School name
- Graduation year (or expected)
- Relevant coursework (optional)

7 Certifications

- List relevant certifications
- Include the issuing organization
- Add the year (optional)
- Only list certifications you have

JANE DOE

Junior DevOps Engineer

jane.doe@example.com | +1 234 567 890 | Toronto, Canada

github.com/janedoe | linkedin.com/in/janedoe | janedoe.dev

PROFESSIONAL SUMMARY

Motivated and detail-oriented aspiring DevOps engineer with hands-on experience in Linux, AWS, Docker, Kubernetes, and CI/CD. Passionate about building reliable, scalable infrastructure and automating deployment processes. Looking for an opportunity to contribute, learn, and grow in a DevOps or cloud engineering role.

TECHNICAL SKILLS

Operating Systems	Linux (Ubuntu, CentOS)
Cloud	AWS (EC2, S3, IAM, VPC)
Containers	Docker, Kubernetes
CI/CD	GitHub Actions, GitLab CI, Jenkins (basic)
Infrastructure as Code	Terraform (basic)
Scripting	Bash, Python (basic)
Monitoring & Logging	Prometheus, Grafana (basic)
Version Control	Git, GitHub

PROJECTS

1. Containerized Web Application with CI/CD
- Built a simple web application and containerized it with Docker
- Set up CI/CD using GitHub Actions for automated testing and deployment
- Deployed to AWS EC2 and used Nginx as a reverse proxy
- GitHub: github.com/janedoe/devops-webapp

2. Terraform Infrastructure on AWS
- Provisioned a VPC, EC2 instance, and security groups using Terraform
- Automated infrastructure deployment and teardown
- Documented the setup and lessons learned
- GitHub: github.com/janedoe/terraform-aws

WORK EXPERIENCE

IT Support Intern | ABC Company | Jun 2024 – Aug 2024
- Supported Linux servers and user accounts
- Assisted with system monitoring and troubleshooting
- Gained exposure to cloud infrastructure and basic automation tasks

EDUCATION

BSc Computer Science | University of ABC | 2020 – 2024
Relevant coursework: Operating Systems, Networking, Cloud Computing

CERTIFICATIONS

- **AWS Certified Cloud Practitioner** | Amazon Web Services | 2024
- **Linux Essentials** | Linux Professional Institute | 2023

Key Tips

- ✓ Keep it one page (for junior roles).
- ✓ Use a clean, readable format.
- ✓ Be honest and specific.
- ✓ Focus on real skills and projects.
- ✓ Use clear action verbs in your experience.
- ✓ Tailor your resume for each job application.

Common Mistakes

- ✗ Listing tools you have never used.
- ✗ Using a generic summary.
- ✗ No real projects or links.
- ✗ Overcrowding with unnecessary details.
- ✗ Poor formatting that ATS cannot parse.
- ✗ Spelling and grammar errors.

Practical Focus

1. Use the template above as a starting point.
2. Replace the example with your own details.
3. Keep it truthful and relevant to the role.
4. Make sure every claimed skill has evidence (project, course, or experience).

Write a Summary You Can Defend

Replace vague statements such as "passionate DevOps professional" with a concise summary of actual skills, engineering work, and target role.

What a Resume Summary Should Do

- ✓ Show who you are and your target role
- ✓ Highlight your key technical skills
- ✓ Reference real projects or experience
- ✓ Be specific, concise, and truthful
- ✓ Match the job you are applying for

Common Mistakes

- ✗ Vague statements (e.g. "passionate")
- ✗ Listing too many tools without context
- ✗ Claiming experience you do not have
- ✗ Making it too long
- ✗ Using generic summaries for every job

A Good Summary Includes

1. Your current stage (beginner, career changer, junior)
2. Key technical skills
3. Real projects or relevant experience
4. What you are looking for
5. Value you can bring to the team

1 Beginner (No Professional DevOps Job Yet)

✗ Weak Summary (Vague)

Passionate DevOps professional looking for an opportunity to grow and contribute to a dynamic team.

✓ Strong, Defensible Summary

Aspiring DevOps engineer with hands-on experience in Linux, Git, Docker, and CI/CD from personal and course projects. Built and deployed containerized applications, automated workflows with GitHub Actions, and managed cloud infrastructure on AWS. Looking for a junior DevOps role to apply my skills, learn from experienced engineers, and contribute to real-world projects.

Why It's Better



- Shows current stage (aspiring)
- Lists key technical skills
- Mentions real projects and tools
- States target role clearly
- Honest and specific

2 Career Changer (From Another Field)

✗ Weak Summary (Vague)

I am a dedicated professional with strong problem-solving skills, passionate about DevOps and cloud technologies, seeking a role to start my career.

✓ Strong, Defensible Summary

Transitioning from a background in [previous field] to DevOps, with hands-on experience in Linux, AWS, Docker, Kubernetes, and infrastructure automation. Completed multiple projects, including a containerized application with CI/CD and a Terraform-based AWS environment. Bringing strong analytical skills, discipline, and a proven ability to learn quickly. Seeking a junior DevOps or cloud engineering role to contribute and grow in a collaborative team.

Why It's Better



- Clearly states career transition
- Highlights relevant technical skills
- References real projects
- Shows transferable skills
- Explains what you can bring
- Sets a clear target role

3 Junior Engineer (Some Relevant Experience)

✗ Weak Summary (Vague)

Junior DevOps engineer with experience in Linux, AWS, Docker, and Kubernetes. Eager to join a challenging team and grow my career.

✓ Strong, Defensible Summary

Junior DevOps engineer with 1+ years of hands-on experience in Linux administration, AWS, Docker, Kubernetes, and CI/CD. Worked on deploying and maintaining containerized applications, implementing infrrastructure with Terraform, and monitoring systems using Prometheus and Grafana. Experienced in troubleshooting issues, automating tasks with Bash and Python, and working in Git-based teams. Looking to contribute to a DevOps or platform engineering team focused on reliable, scalable infrastructure.

Why It's Better



- Includes years of relevant experience
- Gives specific examples of work
- Shows a wider set of skills
- Mentions teamwork and troubleshooting
- States target role and value clearly

Practical Focus: Your Turn



1. Write your own resume summary (1 to 3 sentences).
2. Make sure it is specific and truthful.
3. Get feedback and refine it.
4. Tailor it for the job you are applying for.

Key Takeaways



- ✓ Your summary is your first impression.
- ✓ Be specific, honest, and concise.
- ✓ Use real skills and experience you can defend.
- ✓ Tailor it for each job application.
- ✓ A strong summary increases your chance of getting an interview.

Stop Listing Tools. Start Proving Skills.

Learn to organize Linux, Git, Bash, cloud, Docker, Kubernetes, CI/CD, Terraform, and observability skills without implying expertise you do not have.

1. Don't Just List Tools

- ✗ Listing many tools with no context.
- ✗ It does not show real ability.
- ✗ Looks like keyword stuffing.
- ✗ You may be asked questions you cannot answer.

2. Show Evidence Instead

- ✓ Connect each skill to a project, command, configuration, or troubleshootiing example.
- ✓ Show what you actually did.
- ✓ Keep it honest and realistic for your level.
- ✓ Use clear and specific examples.

3. Be Clear About Your Level

Basic	Intermediate	Advanced
I can follow steps and understand the basics.	I can work independently on common tasks.	I can design, automate, and troubleshoot complex environments.

Skill / Tool	How to Show It on Your Resume	Example (Command / Config / Project)	What This Proves

Linux	Managed users, permissions, services, and logs on Ubuntu during a web server project.	`sudo systemctl status nginx`
		`sudo journalctl -u nginx -n 50`
		`ls -l /var/log/nginx/`
			Ability to manage and troubleshoot Linux servers.

Git	Used Git for version control in all projects with feature branches and pull requests.	`git branch -a`
		`git log --oneline`
		`git push origin feature/login`
			Real experience with version control and collaborative workflows.

Bash	Wrote shell scripts to automate backups and deployment tasks.	`#!/bin/bash` `tar -czf backup.tar.gz /var/www`	Ability to automate repetitive tasks.

Cloud (AWS)	Deployed a web application on AWS using EC2, S3, IAM, and a VPC.	`aws ec2 describe-instances`
		`aws s3 ls`
		`aws iam list-users`
			Hands-on experience managing cloud resources.

Docker	Containerized a web application and used Docker Compose for local development.	`docker build -t myapp .`
		`docker compose up -d`
		`docker ps`
			Ability to build and run containerized applications.

Kubernetes	Deployed an application using Pods, Deployments, and Services on a local or cloud cluster.	`kubectl get pods`
		`kubectl apply -f deployment.yaml`
		`kubectl get services`
			Understanding of core Kubernetes resources and deployment workflows.

CI/CD (GitHub Actions)	Built a CI/CD pipeline to test and deploy a Dockerized application.	`- name: Build and Test` `run: docker build -t myapp .`	Ability to automate testing and deployment.

Terraform	Provisioned AWS infrastructure using Terraform with reusable configuration files.	`terraform init`
		`terraform plan`
		`terraform apply`
			Infrastructure as code and environment provisioming skills.

Observability (Prometheus/ Grafana)	Monitored application metrics and logs using Prometheus and Grafana.	`docker compose up -d prometheus` `docker compose up -d grafana`	Ability to set up monitoring and visualize system health.

Practical Focus: Your Turn

1. Choose 3–5 key skills you are comfortable with.
2. Write one real example for each skill.
3. Include a command, configuration, or project link.
4. Make sure the example matches your actual experience.

Example Resume Bullet

"Deployed a containerized web application on AWS using Docker, Terraform, and GitHub Actions. Configured monitoring with Prometheus and Grafana and automated deployments with a CI/CD pipeline."

Key Takeaways

- ✓ List skills with evidence.
- ✓ Use real projects and examples.
- ✓ Be honest about your level.
- ✓ Focus on what you can actually do.
- ✓ Quality is better than a long list.

No DevOps Job Yet? Show Real Work.

Distinguish personal labs, team projects, internships, volunteering, and paid experience. Learn how to present each honestly without labeling practice environments as production employment.

1. Different Types of Experience

Personal Labs	Projects you build on your own to learn and practice. Valid and valuable, but not paid employment.
Team Projects	Projects you build with others (e.g. open source, study group, hackathon). Shows collaboration and real-world workflows.
Internships	Structured learning with an organization, usually time-bound. Can be paid or unpaid.
Volunteering	You provide technical support for a non-profit, community, or open source project.
Paid Experience	Full-time or part-time employment where you are paid to do the work.

2. How to Present Each One Honestly

- ✓ Use a clear and accurate title (e.g. Personal Project, Volunteer, DevOps Intern).
- ✓ Describe what you built, your role, and the technologies you used.
- ✓ Focus on skills, engineering tasks, and what you learned.
- ✓ Be specific about the environment (e.g. local lab, AWS free tier, non-profit).
- ✓ Do not label a personal lab or training environment as "production" or paid work.
- ✓ Include links to your GitHub repository or project documentation.

3. What to Avoid

- ✗ Calling a personal project a professional job.
- ✗ Using fake job titles.
- ✗ Claiming production experience you do not have.
- ✗ Exaggerating your role or impact.
- ✗ Listing tools you barely used.
- ✗ Copying project descriptions from others.
- ✗ Hiding that it was a training, lab, or personal project.

4. Real Examples for Each Type

Personal Lab	• Set up a Linux server • Deployed a web application with Docker • Documented the steps in a GitHub repo
Team Project	• Built a CI/CD pipeline with a study group • Used GitHub Actions and Docker • Collaborated using pull requests
Internship	• Assisted in infrastructure monitoring • Worked with Linux and AWS • Learned incident response and documentation
Volunteering	• Managed a website for a non-profit • Set up backups and monitoring • Improved deployment process
Paid Experience	• Managed cloud infrastructure • Automated deployments with CI/CD • Troubleshot issues and improved reliability

5. Turn a Training Project Into a Resume Entry

Project Overview

You built a web application on AWS using Linux, Docker, and a CI/CD pipeline with GitHub Actions.



How to Present It on Your Resume

Personal Project: Deploy a Web Application on AWS | Jan 2024 – Mar 2024

- Built and deployed a containerized web application using Docker on an Amazon EC2 instance (Linux).
- Implemented a CI/CD pipeline with GitHub Actions to automate testing and deployment.
- Configured security groups, IAM roles, and Nginx as a reverse proxy.
- Monitored application logs and uptime using basic logging and monitoring tools (e.g. CloudWatch).
- Documented the full setup, steps, and troubleshooting in a GitHub repository: github.com/your-username/devops-webapp

Practical Focus: Your Turn

1. Choose a completed Linux or AWS training project.
2. Identify the skills and technologies used.
3. Write 3–5 bullet points for your resume.
4. Add a link to your GitHub repository.
5. Make sure the description is honest and accurate.

Key Takeaways

- ✓ You do not need a paid DevOps job to have a strong resume.
- ✓ Personal labs and projects are valuable when presented honestly.
- ✓ Focus on real engineering tasks, not just tools.
- ✓ Provide evidence (GitHub links, documentation).
- ✓ Be clear about your role and environment.

Pro Tip

A well-documented personal project with clear steps, code, and real configurations can be more impressive than a vague job description.

The Engineering Bullet Formula

Write evidence-based bullets using action, system, implementation, verification, and outcome. Distinguish measured results from assumptions and avoid invented performance improvements.

1. The Bullet Formula

A strong engineering bullet tells a complete story.
Use this simple structure:

Action	System	Implementation	Verification	Outcome
What you did.	Where you did it.	How you did it.	How you confirmed it worked.	What was the result (real and measurable).

2. Key Guidelines

- ✓ Be specific and technical.
- ✓ Use real tools, commands, and technologies.
- ✓ Mention what you actually did, not what the team did.
- ✓ Include how you verified the work.
- ✓ State real results. If you don't have numbers, describe the actual impact.
- ✓ Be honest about your level and avoid exaggeration.
- ✓ Use past tense for completed work.

3. Weak vs Strong Examples

✗ Weak Bullet (Vague)	✓ Strong Bullet (Defensible)
Worked with Docker.	Built and deployed a containerized web application using Docker and Docker Compose, verified the containers were running with docker ps, and documented the setup in a GitHub repository.
Used AWS.	Provisioned a Linux EC2 instance on AWS, configured security groups and IAM roles, and verified connectivity using SSH and AWS CLI.
Worked on CI/CD.	Built a CI/CD pipeline using GitHub Actions to automatically build, test, and deploy a Dockerized application, verified successful deployments in a test environment.
Used Kubernetes.	Deployed a containerized application on a local Kubernetes cluster using Deployments and Services, verified pod health with kubectl, and tested rolling updates.
Worked with Terraform.	Created reusable Terraform configuration files to provision an AWS VPC, subnets, and an EC2 instance, verified the resources with terraform plan and terraform apply.
Used monitoring tools.	Set up application and system monitoring using Prometheus and Grafana, verified metrics collection, and created dashboards to visualize CPU, memory, and HTTP request metrics.

4. Break Down the Example: Docker

Weak Bullet

✗ Worked with Docker.

Strong Bullet (Step by Step)

Action	Built and deployed
System	a containerized web application
Implementation	using Docker and Docker Compose
Verification	verified the containers were running with docker ps
Outcome	and documented the setup in a GitHub repository.

5. More Examples for Common Skills

Skill	Defensible Bullet Example
Linux	Configured and managed a Linux server on Ubuntu, set up a web application, and troubleshot service issues using systemctl and journalctl.
AWS	Provisioned an EC2 instance, configured security groups and IAM roles, and verified connectivity using the AWS CLI.
Git	Used Git for version control with feature branches and pull requests, and maintained a clean commit history.
Bash	Wrote Bash scripts to automate backups, verified successful execution, and scheduled them with cron.

6. Measured Results vs Assumptions

✓ Real and Acceptable

- ✓ Reduced deployment time from 30 minutes to 10 minutes (measured).
- ✓ Improved setup reliability by automating configuration (observed and verified).
- ✓ Supported a team of 5 developers (internship/project/volunteer role).
- ✓ Deployed a test environment used for learning and demonstrations.

✗ Avoid These Assumptions

- ✗ Increased performance by 70% (no evidence).
- ✗ Handled millions of users (not true).
- ✗ Led a DevOps team (if you did not).
- ✗ Improved security significantly (be specific).
- ✗ Used in production (if it was a personal lab).

7. Practical Focus: Your Turn

1. Take a vague bullet like "Worked with Docker."
2. Rewrite it using the bullet formula.
3. Do the same for one other skill (e.g. AWS, Linux, or CI/CD).
4. Make sure each bullet is honest, specific, and technically accurate.

8. Key Takeaways

- ✓ A strong bullet shows real engineering work.
- ✓ Always include action, system, implementation, verification, and outcome.
- ✓ Use real tools, commands, and examples.
- ✓ Be honest about your level and environment.
- ✓ Quality is better than a long list of vague bullets.

VERIQTA

Three Projects Worth Putting on a Resume

Study three portfolio-worthy engineering scenarios: Linux service operations, a containerized application with CI/CD, and a Terraform-managed cloud environment.
Identify the architecture, implementation evidence, failure testing, and verification each project needs before it earns resume space.

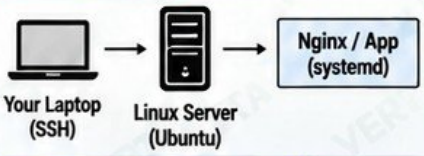
1 Linux Service Operations

Manage and troubleshoot a real service on a Linux server.

Project Scenario

Set up a Linux server, deploy a web application (e.g. Nginx or a simple Python app), configure it as a systemd service, manage users, permissions, logs, and monitoring, and practice troubleshooting common failures.

Architecture



Key Implementation Steps

1. Provision a Linux server (local VM or cloud).
2. Install and configure Nginx or a Python app.
3. Create and manage a systemd service.
4. Configure firewall and basic security.
5. Set up logging and monitoring (e.g. journalctl, log files).

Failure Testing

- Stop the service and verify it fails.
- Check logs for errors.
- Simulate high CPU or memory usage.
- Test firewall rules (block/allow ports).
- Fix permissions or configuration mistakes.

Verification Evidence

- ✓ Screenshots of commands and outputs.
- ✓ Service status (systemctl status).
- ✓ App accessible in browser.
- ✓ Logs showing the service running.
- ✓ A README with setup steps and troubleshooting.

Resume Bullet Example

Set up and managed an Nginx web server on Ubuntu, configured it as sa systemd service, implemented firewall rules, and monitored logs using journalctl, documenting the full setup and troubleshooting process in a GitHub repository.

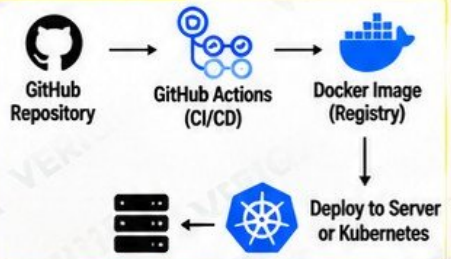
2 Containerized Application with CI/CD

Build, test, and deploy a containerized application using a CI/CD pipeline.

Project Scenario

Containerize a web application, use GitHub Actions to build and test the image, and deploy it to a server or Kubernetes. Include automated testing and deployment.

Architecture



Key Implementation Steps

1. Create a simple web application (e.g. Node.js or Python).
2. Write a Dockerfile and build the image.
3. Set up GitHub Actions for build, test, and push.
4. Deploy to a Linux server or Kubernetes.
5. Use environment variables and secrets securely.

Failure Testing

- Introduce a failing test and check pipeline failure.
- Use a wrong image tag and verify rollback.
- Break the application (e.g. bad config) and verify the pipeline catches it.
- Test container restarts and health checks.

Verification Evidence

- ✓ GitHub repository with code and workflow files.
- ✓ Successful and failed pipeline runs (screenshots).
- ✓ Docker image in registry (e.g. Docker Hub or GHCR).
- ✓ Deployed application accessible via browser.
- ✓ README with setup, testing, and rollback steps.

Resume Bullet Example

Built and deployed a containerized web application using Docker and GitHub Actions, implementing a CI/CD pipeline with automated testing, and deployed the application to a Linux server, documenting the full process and failure scenarios in a GitHub repository.

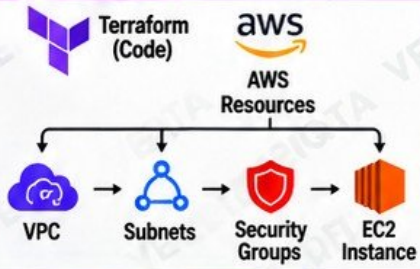
3 Terraform-Managed Cloud Environment

Use Infrastructure as Code to provision and manage cloud resources.

Project Scenario

Use Terraform to provision a VPC, subnets, security groups, and an EC2 instance (or a small application environment) on AWS. Keep the configuration modular and reusable.

Architecture



Key Implementation Steps

1. Install and configure Terraform.
2. Write modular Terraform code (VPC, subnets, SG, EC2).
3. Use variables and outputs.
4. Run terraform init, plan, and apply.
5. Test connectivity (SSH) and verify resources.

Failure Testing

- Introduce a misconfiguration and check plan output.
- Try to deploy with missing variables.
- Test destroying and recreating the environment.
- Verify security group rules by allowing and blocking traffic.

Verification Evidence

- ✓ Terraform code in a GitHub repository.
- ✓ Screenshots of terraform plan and apply.
- ✓ AWS console showing the provisioned resources.
- ✓ SSH into the instance and run basic commands.
- ✓ README with setup, teardown, and cost notes.

Resume Bullet Example

Created a reusable Terraform configuration to provision an AWS VPC, subnets, security groups, and an EC2 instance, validated the deployment, tested connectivity, and documented the full infrastructure setup and teardown process in a GitHub repository.

Practical Focus: Your Turn

1. Build one project from each category.
2. Document the architecture and implementation steps.
3. Perform at least two failure tests per project.
4. Capture verification evidence (screenshots, logs, outputs).
5. Write a resume bullet for each project.

Key Takeaways

- ✓ Real projects show real skills.
- ✓ Include architecture, commands, and configuration.
- ✓ Demonstrate troubleshooting and failure testing.
- ✓ Keep your documentation clear and organized.
- ✓ Quality and honesty are more important than quantity.

Pro Tip

Use a public GitHub repository with a clear README. Employers often review your code and setup steps to see how you think, not just what tools you listed.

Your GitHub Is Part of the Interview

Learn what a technical reviewer should find in a project repository: a clear README, architecture diagram, setup steps, configuration examples, testing evidence, limitations, and cleanup instructions.

1. What Reviewers Look For

- ✅ Clear and well-organized README.
- ✅ Real, working code and configuration files.
- ✅ Architecture diagram that shows the big picture.
- ✅ Step-by-step setup and run instructions.
- ✅ Evidence of testing (screenshots, logs, or test results).
- ✅ Explanation of design decisions and trade-offs.
- ✅ Known limitations and possible improvements.
- ✅ No exposed secrets (keys, passwords, tokens).
- ✅ Clean, professional, and easy to navigate.

2. A Good Repository Structure

your-project-name/	
README.md	Project overview, setup, and usage
docs/	Architecture diagram and details
src/	Application or scripts
infrastructure/	Terraform, Kubernetes, or cloud configs
configs/	Example configuration files
scripts/	Helper scripts (setup, deploy, test)
tests/	Test files and evidence
.github/	CI/CD workflow files
.gitignore	Exclude unnecessary files
LICENSE	License for your project

3. README.md Must Include

README.md

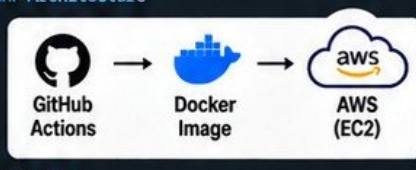
- **Project Title and Brief Description** — What the project does and why you built it.
- **Architecture Diagram** — Visual diagram of the components.
- **Prerequisites** — Tools and accounts needed.
- **Setup Steps** — Step-by-step instructions to run the project.
- **Configuration Examples** — Environment variables and sample config files.
- **How to Test** — Commands to test and expected output.
- **Screenshots / Demo** — Proof that it works (screenshots, logs, or video).
- **Limitations** — What is not included or could be improved.
- **Cleanup Instructions** — How to remove resources (especially cloud).

4. Example README Sections

```
# DevOps Project: Nginx on AWS with CI/CD

## ProjOps Overview
A containerized web application deployed to AWS
using Terraform and GitHub Actions.

## Architecture
```




```
## Setup
git clone https://github.com/username/project.git
cd project
./scripts/setup.sh

## Configuration
cp .env.example .env
# Update the values in .env

## Testing
curl http://your-ec2-public-ip

## Cleanup
terraform destroy
```

5. Good vs Bad Examples

Good README (Clear and Complete)

- ✅ Explains the project and its purpose.
- ✅ Includes an architecture diagram.
- ✅ Provides step-by-step setup instructions.
- ✅ Shows real configuration examples.
- ✅ Includes testing evidence (screenshots/logs).
- ✅ Mentions limitations and future improvements.
- ✅ Includes cleanup steps.

Bad README (Missing Important Details)

- ❌ Vague or no description.
- ❌ No architecture diagram.
- ❌ Missing or unclear setup steps.
- ❌ No configuration examples.
- ❌ No proof that it works.
- ❌ No limitations or next steps.
- ❌ No cleanup instructions.

6. Audit a Repository (Checklist)

- ✅ Can you understand what the project does?
- ✅ Can you reproduce it from scratch?
- ✅ Are the instructions clear and accurate?
- ✅ Is the code organized and easy to follow?
- ✅ Are there any exposed secrets or sensitive data?
- ✅ Is there evidence that it was tested?
- ✅ Does it show real learning and effort?
- ✅ Would you be comfortable discussing it in an interview?

7. Common Security Mistakes to Avoid

- ❌ Committing API keys or cloud credentials.
- ❌ Hardcoding secrets in configuration files.
- ❌ Leaving .env files in the repository.
- ❌ Exposing database passwords.
- ❌ Sharing sensitive information in screenshots.
- ❌ Not using .gitignore to exclude secret files.

8. Practical Focus: Your Turn

1. Choose a sample repository (yours or a public one).
2. Check if all README sections are present.
3. Try to follow the setup steps.
4. Look for any exposed secrets.
5. Identify what makes the repository credible and what needs improvement.

9. Key Takeaways

- ✅ Your GitHub shows how you work, not just what you know.
- ✅ A clear README and working project builds trust.
- ✅ Good documentation makes your project stand out.
- ✅ Security, reproducibility, and honesty matter.
- ✅ Treat every repository as part of the interview.

Tailor Your Resume Without Lying

Match a resume to an actual job description. Separate required technologies, responsibilities, transferable skills, and missing experience.
Practical focus: Create two truthful resume variations from one evidence inventory, one for junior cloud engineering and another for DevOps.

1. Understand the Job Description

Sample Job Description
Junior Cloud Engineer

- Manage and maintain cloud infrastructure on AWS (EC2, S3, IAM, VPC).
- Work with infrastructure as code (Terraform).
- Monitor systems and troubleshoot issues.
- Collaborate with development teams.
- Write documentation and follow best practices.
- Nice to have: Docker, Kubernetes, GitHub Actions.

2. Break It Down

Required Technologies
AWS, Terraform, Linux

Key Responsibilities
Manage, monitor, troubleshoot, document

Transferable Skills
Linux, scripting, teamwork, documentation

Missing Experience
Kubernetes (not required), production environment (experience preferred)

3. Your Evidence Inventory

- ✓ Linux server setup and administration
- ✓ AWS (EC2, S3, IAM, VPC) labs and projects
- ✓ Terraform infrastructure project
- ✓ Docker containerized application
- ✓ GitHub Actions CI/CD pipeline
- ✓ Monitoring with Prometheus and Grafana
- ✓ Documentation (README, diagrams)
- ✓ Troubleshooting and failure testing
- ✓ Bash scripting and automation
- ✓ Team project collaboration

4. Map Your Evidence to the Job

Job Requirement	Your Evidence	Use in Resume?
AWS (EC2, S3, IAM, VPC)	Terraform AWS project, cloud labs	✓
Terraform	Infrastructure as code project	✓
Linux administration	Server setup, systemd, troubleshooting	✓
Monitoring	Prometheus, Grafana project	✓
CI/CD	GitHub Actions pipeline	✓
Docker	Containerized application project	✓
Kubernetes	Kubernetes lab (basic)	!
Production experience	Not yet (lab environment)	✗

5. Resume Variation 1 (Junior Cloud Engineer)

ALEX JOHNSON
Junior Cloud Engineer
alex.johnson@email.com | linkedin.com/in/alexjohnson
github.com/alexjohnson | City, Country

Professional Summary

Junior Cloud Engineer with hands-on experience in AWS, Terraform, and Linux. Experienced in provisioning cloud infrastructure, automating deployments, and monitoring systems through personal and team projects. Passionate about building reliable and scalable cloud solutions.

Key Skills

AWS (EC2, S3, IAM, VPC) | Terraform | Linux
Bash | Git | Cloud Monitoring (Prometheus, Grafana)

Selected Projects

- Provisioned a VPC, subnets, and EC2 instances on AWS using Terraform.
- Deployed a web application on AWS with automated deployment using GitHub Actions.
- Set up monitoring with Prometheus and Grafana.

Education & Certifications

- B.Sc. Computer Science (In View)
- AWS Cloud Practitioner (In Progress)

6. Resume Variation 2 (DevOps)

ALEX JOHNSON
Junior DevOps Engineer
alex.johnson@email.com | linkedin.com/in/alexjohnson
github.com/alexjohnson | City, Country

Professional Summary

Junior DevOps Engineer with hands-on experience in Linux, Docker, CI/CD, and infrastructure as code. Skilled in building, testing, and deploying containerized applications using GitHub Actions and Terraform. Strong focus on automation, monitoring, and troubleshooting in cloud and on-prem environments.

Key Skills

Linux | Docker | Kubernetes (basic) | CI/CD
GitHub Actions | Terraform | AWS | Bash
Monitoring (Prometheus, Grafana)

Selected Projects

- Built and deployed a containerized application using Docker and GitHub Actions.
- Created a Terraform project to provision AWS infrastructure.
- Implemented monitoring and alerting with Prometheus and Grafana.

Education & Certifications

- B.Sc. Computer Science (In View)
- AWS Cloud Practitioner (In Progress)

7. What Not to Do

- ✗ Do not list technologies you have never used.
- ✗ Do not claim production experience for lab projects.
- ✗ Do not copy the job description word for word.
- ✗ Do not exaggerate your level or impact.
- ✗ Do not hide gaps. Be honest and focus on what you can do now.

Practical Focus: Your Turn

1. Use your own evidence inventory.
2. Choose a real job description (cloud or DevOps).
3. Identify required, preferred, and missing items.
4. Create two resume variations from the same evidence.
5. Make sure both are truthful, focused, and tailored.

9. Key Takeaways

- ✓ Tailoring means highlighting relevant skills, not inventing new ones.
- ✓ Use the same evidence to create different resume versions for different roles.
- ✓ Be honest about your level of experience.
- ✓ Focus on real projects, real skills, and transferable experience.
- ✓ A tailored resume shows you understand the role and increases your chances of an interview.

10. Pro Tip

Keep a master resume with all your skills and projects. Create role-specific versions by selecting and reordering the most relevant sections for each job description.

The 60-Second DevOps Introduction

Learn how to answer "Tell me about yourself" without reciting your resume. Connect your background, strongest technical evidence, and career direction.

Practical focus: Build a credible 60-second introduction and prepare for the first follow-up question.

1. What Interviewers Want to Hear

- A clear and concise story (not your life story).
- Relevant technical skills and real evidence.
- Why you are interested in DevOps and where you are heading.
- Confidence, honesty, and good communication.
- Not a word-for-word repetition of your resume.

2. A Simple Structure to Follow

1. **Background (10–15 seconds)**
   Who you are and your relevant experience or transition.
2. **Key Technical Evidence (20–25 seconds)**
   Highlight 1–2 real projects, skills, or achievements.
3. **Career Direction (10–15 seconds)**
   What you are aiming for and why.
4. **Closing (5–10 seconds)**
   Summarize your value and openness to learn.

3. Do's and Don'ts

Do

- Keep it to about 60 seconds.
- Be specific and mention real examples.
- Show enthusiasm and clarity.
- Connect your skills to the role.
- Practice until it sounds natural.

Don't

- Don't recite your resume.
- Don't exaggerate or make untrue claims.
- Don't include unrelated personal details.
- Don't speak for too long.

4. Example Introductions for Different Backgrounds

A. Beginner (New to Tech)



"I'm transitioning into DevOps from a non-technical background. I've built hands-on skills in Linux, Git, Docker, and AWS through personal projects. For example, I deployed a containerized web application on AWS using GitHub Actions. I'm excited to continue growing in DevOps and contribute to a team where I can learn, solve real problems, and build reliable systems."

B. Career Changer (Related Field)



"I have a background in IT support, and I've expanded into DevOps by building skills in Linux administration, AWS, Terraform, and CI/CD. I've completed projects such as provisioning AWS infrastructure with Terraform and setting up a CI/CD pipeline for a Dockerized application. I'm now focused on Cloud/DevOps roles where I can apply my infrastructure and automation skills."

C. Junior Engineer (Some Experience)



"I'm a junior DevOps engineer with hands-on experience working with Linux, Docker, Kubernetes, and AWS. I've built and deployed applications using CI/CD pipelines with GitHub Actions, and I've worked on infrastructure as code with Terraform. I enjoy automating processes, solving infrastructure problems, and I'm looking to grow in a collaborative team environment."

5. Break Down the Example (Beginner)

"I'm transitioning into DevOps from a non-technical background. I've built hands-on skills in Linux, Git, Docker, and AWS through personal projects. For example, I deployed a containerized web application on AWS using GitHub Actions. I'm excited to continue growing in DevOps and contribute to a team where I can learn, solve real problems, and build reliable systems."

Background	States current situation and transition.
Technical Evidence	Mentions specific skills and a real project.
Career Direction	Shows goals and enthusiasm.
Closing	Ends with value and willingness to learn.

6. Common Follow-Up Questions (with Tips)

Q1	Why are you interested in DevOps?	Talk about what excites you (automation, problem-solving, impact) and connect it to your background.
Q2	Can you tell me more about one of your projects?	Be ready to explain the goal, tools used, your role, challenges, and what you learned.
Q3	What are you still learning?	Be honest and mention specific technologies or concepts you are currently working on.
Q4	How do you handle challenges?	Give a real example of a problem you faced and how you approached it.

Practical Focus: Your Turn

1. Write your own 60-second introduction.
2. Record yourself and check the time.
3. Practice answering the first follow-up question.
4. Refine it until it sounds natural and confident.

8. Key Takeaways

- Keep it clear, concise, and relevant.
- Use real technical evidence.
- Show your career direction.
- Be honest and authentic.
- Practice until it feels natural.
- Be ready for follow-up questions.

9. Pro Tip



Time yourself. Aim for about 60 seconds. If it's much longer, remove extra details. If it's much shorter, add one more relevant example.

Linux and Networking Under Pressure

Prepare for questions about permissions, processes, services, logs, DNS, ports, and connectivity.

Interview scenario: An application is unreachable. Explain how systemctl, ss, curl, and journalctl help investigate different possible causes, rather than guessing the fix.

1. The Scenario

You deployed a web application on a Linux server, but it is unreachable.
Users report: "The site is not loading."

Your job:

- Investigate methodically using the right tools.
- Find the actual cause (don't guess).
- Explain what each command tells you.
- Consider multiple possible causes (service down, wrong port, firewall, DNS, etc.).

2. Troubleshooting Flow (High Level)



3. Linux Permissions (Quick Checks)

Check file permissions (common in web apps)

```
$ ls -l /var/www/html
$ chmod 755 /var/www/html
$ chown -R www-data:www-data /var/www/html
```

Check running process user

```
$ ps aux | grep nginx
$ ps aux | grep your-app
```

Common Interview Points

- What permissions are required for web files?
- How do you change file ownership?
- Why might wrong permissions cause 403/500 errors?

4. Check the Service (systemctl)

Verify the application or web server is running.

```
$ systemctl status nginx
  # Check if the service is active, failed, or exited
$ systemctl start nginx
$ systemctl restart nginx
$ systemctl enable nginx
```

What to look for:

- Is the service active (running)?
- Any error messages?
- Is it enabled to start on boot?

5. Check Open Ports (ss)

Verify if the application is listening on the expected port.

```
$ ss -tuln | grep :80
$ ss -tuln | grep :443
$ ss -tuln | grep :8080
```

What to look for:

- Is the application listening on the correct port?
- Is it bound to 0.0.0.0 (external access) or 127.0.0.1 (only local)?
- Is the port closed or used by another process?

6. Test Connectivity (curl)

Test if the application responds locally and remotely.

```
$ curl -v http://localhost
$ curl -v http://localhost:8080
$ curl -v http://your-server-ip
$ curl -v http://your-domain.com
```

What to look for:

- Do you get a response (200, 301, 403, 500)?
- Connection refused (service not running)?
- Timeout (firewall, network, or security group issue)?
- DNS resolution working (if using a domain)?

7. Check the Logs (journalctl)

Find detailed error messages from the service.

```
$ journalctl -u nginx -n 50
$ journalctl -u nginx -f
$ journalctl -u your-app -n 50
```

What to look for:

- Application startup errors
- Configuration errors
- Permission issues
- Port binding errors
- Crash or out-of-memory errors

8. Other Possible Causes to Consider

DNS Issue	Check if the domain resolves to the correct IP.	`$ nslookup your-domain.com` `$ dig your-domain.com`
Firewall	Check if the port is blocked.	`$ sudo ufw status` `$ sudo iptables -L -n`
Cloud Security Group	Ensure the port is allowed (e.g. 80, 443). Check in your cloud provider console (AWS, Azure, GCP).	
Network Connectivity	Test from another server or your local machine using curl or telnet. `$ telnet your-server-ip 80`	
Configuration Issue	Wrong port, wrong bind address, misconfigured reverse proxy, etc.	

9. Example Investigation Output

```
# 1. Service status
$ systemctl status nginx
● nginx.service - active (running)

# 2. Port check
$ ss -tuln | grep :80
tcp    LISTEN  0    128    0.0.0.0:80

# 3. Connectivity test
$ curl -I http://localhost
HTTP/1.1 200 OK

# 4. Logs (if there was an error)
$ journalctl -u nginx -n 5
... [error] failed to bind to port 80:
    Address already in use
```

10. Practical Focus: Your Turn

1. Investigate a real or simulated "application unreachable" scenario on a Linux server.
2. Use systemctl, ss, curl, and journalctl to find the cause.
3. Document what you found and how you fixed it.
4. Prepare to explain your investigation steps in an interview.

11. Key Takeaways

- Don't guess. Investigate systematically.
- Use the right tool for the right question.
- Each command gives different information.
- Consider multiple possible causes.
- Explain your thought process, not just the fix.
- This is real-world problem solving, not memorizing commands.

12. Pro Tip

In an interview, talk through your investigation step by step. State what you expect to see, what you see, and what it means. This shows structured thinking and real engineering skills.

Git, CI/CD, and Failed Deployments

Understand what interviewers expect when asking about branches, pull requests, automated testing, build artifacts, deployment failures, and rollback decisions.

Interview scenario: A deployment succeeds, but application health checks fail. Explain your investigation and the evidence that would justify rollback.

1. Git Basics Interviewers Ask About

Branches
Used to develop features, fix issues, and keep work isolated.

Pull Requests
Used to review code, run automated checks, and merge safely.

Code Review
Helps catch issues early and maintain code quality.

Merge Strategies
merge, squash, or rebase (know the difference).

2. Typical CI/CD Pipeline Flow

Code Push
(to main or via Pull Request)
↓
Automated Tests
(unit, integration, linting)
↓
Build Artifact
(e.g. Docker image, packaged app)
↓
Deploy to Environment
(dev → staging → production)
↓
Health Checks
(application and system checks)

3. Key Interview Concepts

- [x] What happens in a pull request?
- [x] How do automated tests prevent bad deployments?
- [x] What is a build artifact? (e.g. Docker image)
- [x] How do you promote an artifact to production?
- [x] How do you handle a failed deployment?
- [x] When would you rollback?
- [x] What evidence do you need before rollback?
- [x] How do you verify the rollback worked?

4. Example CI/CD Pipeline (GitHub Actions)

```yaml
name: CI/CD Pipeline
on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Run Tests
        run: pytest
      - name: Build Docker Image
        run: docker build -t myapp:latest .
      - name: Push to Registry
        run: docker push myapp:latest
      - name: Deploy to Server
        run: ./deploy.sh
```

5. The Scenario: Deployment Succeeds But Health Checks Fail

Code Merged → Pipeline Succeeds → App Deployed → Health Check Fails

Symptoms You Might See

- [ ] Deployment completed with no errors.
- [ ] Application returns 5xx errors.
- [ ] Health check endpoint is failing.
- [ ] Users cannot access the application.
- [ ] Logs show runtime errors.
- [ ] Metrics show high error rate or zero healthy instances.

6. Investigation Steps

1. **Check application health endpoint**
```
curl -v https://your-app.com/health
```
2. **Check application logs**
```
journalctl -u your-app -n 100
```
3. **Check container status (if using Docker)**
```
docker ps
docker logs <container_id>
```
4. **Check Kubernetes deployment (if using K8s)**
```
kubectl get pods
kubectl logs <pod-name>
```
5. **Check recent changes**
```
git log --oneline -n 5
```
6. **Check monitoring/alerts**
Look at metrics (CPU, memory, error rate) and APM logs (e.g. Prometheus, Grafana).

7. Evidence That Justifies a Rollback

- [x] Health check endpoint consistently failing (e.g. 5xx errors).
- [x] Application logs showing new errors after deployment.
- [x] Error rate significantly higher than before.
- [x] New version causing increased latency or timeouts.
- [x] Core functionality not working.
- [x] No quick fix available or high risk to users.

8. How to Rollback (Examples)

Docker (manual rollback)
```
docker pull myapp:previous
docker stop myapp
docker run -d --name myapp myapp:previous
```

Kubernetes (rollback to previous version)
```
kubectl rollout history deployment/myapp
kubectl rollout undo deployment/myapp
```

Git (revert a bad commit)
```
git revert <commit-hash>
git push origin main
```

9. Verify After Rollback

- [x] Health checks return 200 (OK).
- [x] Application is accessible to users.
- [x] Error rate returns to normal.
- [x] Logs show no errors.
- [x] Monitoring dashboards are healthy.
- [x] Confirm with users or team that functionality is restored.

10. Practical Focus: Your Turn

1. Investigate the scenario using the steps above.
2. Identify the most likely cause of the health check failure.
3. Decide if rollback is the right action and explain why.
4. Write a short answer you would give in an interview, including the evidence you used.

11. Key Takeaways

- [x] A successful deployment does not mean the application is working.
- [x] Use logs, health checks, and monitoring to find the cause.
- [x] Rollback decisions should be based on evidence, not panic.
- [x] Be ready to explain your investigation steps clearly.
- [x] Show that you understand both tools and reasoning.

12. Pro Tip

In an interview, walk through your thought process out loud. Show your commands, what you expect to see, and how each result helps you decide the next step.

VERIQTA

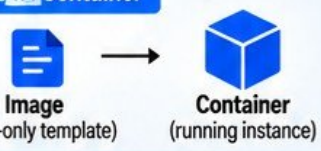
Docker and Kubernetes Interview Traps

Prepare to explain images versus containers, image tags, volumes, Pods, Deployments, Services, and health probes.

Interview scenario: A Kubernetes Pod enters CrashLoopBackOff. Explain what the status means, which evidence to inspect, and why restarting everything is not a reliable diagnosis.

1. Docker Basics Interview Points

Image VS Container



- [x] What is a Docker image?
- [x] What is a container?
- [x] How are image tags used?
- [x] What is the difference between COPY and volume?
- [x] How do volumes persist data?
- [x] How do you troubleshoot a container that exits immediately?

2. Common Docker Commands

```
# Build an image
docker build -t myapp:1.0 .

# Run a container with a volume
docker run -d -p 8080:80 \
  -v app-data:/data myapp:1.0

# List containers
docker ps -a

# View logs
docker logs <container_id>

# Inspect container
docker inspect <container_id>
```

3. Kubernetes Core Concepts

Pod
Smallest deployable unit. Runs one or more containers.

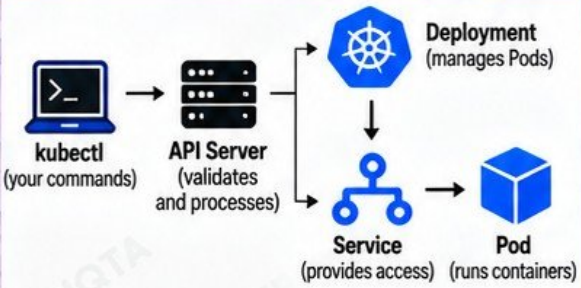
Deployment
Manages replicas and rolling updates.

Service
Provides stable network access to Pods (ClusterIP, NodePort, LoadBalancer).

Volume
Persists data beyond the container lifecycle.

Health Probes
Liveness, readiness, and startup probes keep applications healthy.

4. Kubernetes Architecture (Simplified)



5. The Scenario: Pod in CrashLoopBackOff

STATUS
CrashLoopBackOff

The **Pod** starts, runs for a short time, crashes, and Kubernetes keeps restarting it. This usually means there is a problem with the application, configuration, dependencies, or environment, not just a cluster issue.

6. Evidence to Inspect

1. **Check Pod status and events**
```
kubectl get pods
kubectl describe pod <pod-name>
```
2. **View container logs (current and previous)**
```
kubectl logs <pod-name>
kubectl logs <pod-name> --previous
```
3. **Check resource usage**
```
kubectl top pod <pod-name>
```
4. **Check configuration (env, config, secrets)**
```
kubectl describe pod <pod-name>
```
5. **Verify dependencies and connectivity**
```
kubectl exec -it <pod-name> -- curl <service>
```

7. Common Causes of CrashLoopBackOff

- **Application error** (e.g. exception, missing file, wrong command)
- **Configuration issue** (e.g. incorrect env variables, config file)
- **Missing dependency** (e.g. database not reachable, DNS issue)
- **Permission issue** (e.g. cannot read file, run as non-root)
- **Health probe failure** (e.g. app takes too long to start)
- **Resource limit** (e.g. out of memory, CPU throttling)

8. Why Restarting Everything Is Not the Answer

- ✗ It does not fix application bugs or configuration mistakes.
- ✗ The Pod will keep crashing again.
- ✗ It can hide the real issue and waste time.
- ✗ It does not provide useful evidence.
- ✗ It may cause unnecessary downtime for other healthy components.
- ✓ Investigate systematically using logs, events, and configuration to find the root cause.

9. Example Investigation Output

```
# Pod status
$ kubectl get pods
NAME                READY  STATUS             RESTARTS  AGE
myapp-7d6f9c8c7-abc12 0/1  CrashLoopBackOff   5         2m

# Describe (events)
$ kubectl describe pod myapp-7d6f9c8c7-abc12
Events:
  Type     Reason     Message
  ----     ------     -------
  Warning  BackOff    Back-off restarting
                      failed container

# Logs
$ kubectl logs myapp-7d6f9c8c7-abc12 --previous
Error: Cannot connect to database at db:5432
Connection refused
```

10. Practical Focus: Your Turn

1. Analyze a real or simulated CrashLoopBackOff case.
2. Use the commands above to find the root cause.
3. Explain what you would fix and why.
4. Prepare to discuss this investigation in an interview.

11. Key Takeaways

- [x] Understand the difference between image and container.
- [x] Know how Pods, Deployments, and Services work together.
- [x] Always check logs, events, and configuration.
- [x] Identify the real cause before taking action.
- [x] Restarting is not a solution, it is a temporary workaround.
- [x] Explain your reasoning clearly in an interview.

12. Pro Tip

Practice troubleshooting CrashLoopBackOff with different scenarios (env variable, missing file, database down, bad config). Being able to explain your investigation process is more important than memorizing commands.

Cloud, Terraform, and Security Questions

Prepare for AWS identity and access management, networking, security groups, infrastructure as code, Terraform state, and secrets handling.

Interview scenario: A Terraform plan proposes deleting a cloud resource. Explain how you would inspect the change, identify its impact, and prevent an unsafe deployment.

1. Key AWS Concepts to Know

Identity and Access Management (IAM)
Users, groups, roles, policies, and least privilege access.

Networking
VPC, subnets, route tables, internet gateway, NAT gateway, and private/public subnets.

Security Groups
Stateful firewalls that control inbound and outbound traffic.

Common AWS Services
EC2, S3, RDS, Lambda, CloudWatch, CloudTrail, and VPC.

2. Terraform Core Concepts

Infrastructure as Code (IaC)
Define and manage cloud resources using code.

Terraform Configuration
Providers, resources, variables, outputs, and modules.

Terraform State
Tracks real infrastructure to know what exists.

Plan, Apply, and Destroy
Preview changes (plan), make changes (apply), and remove resources (destroy).

Remote State
Use a secure remote backend (e.g. S3 with DynamoDB locking) for team work.

3. Security Best Practices

Least Privilege Access
Give only the permissions needed.

Secrets Handling
Use AWS Secrets Manager, SSM Parameter Store, or environment variables (never hardcode secrets).

Secure Networking
Use security groups, NACLs, and private subnets.

Audit and Monitoring
Use CloudTrail, CloudWatch, and Config to track changes.

Data Protection
Encrypt data at rest and in transit.

Compliance and Governance
Follow organizational and industry security standards.

4. Terraform Plan: Understanding Changes

```
$ terraform plan
...
# aws_instance.web will be destroyed
- resource "aws_instance" "web" {
    id    = "i-0abc123"
    tags = {
      Name  = "web-server"
    }
  }
Plan: 0 to add, 0 to change, 1 to destroy.
```

5. The Scenario

You run terraform plan and notice that a critical resource (e.g. an EC2 instance, RDS database, or S3 bucket) is marked for deletion. The plan shows 1 to destroy.

How do you investigate, understand the impact, and prevent an unsafe deployment?

6. Investigation Steps

1. **Review the full plan output**
   Identify which resource will be deleted and why.
2. **Check recent code changes**
   Look for modifications, removed code, or changed variables.
3. **Compare with current state**
   Use terraform state show <resource> to see the current configuration.
4. **Check for dependencies**
   Identify what other resources depend on it (e.g. databases, load balancers).
5. **Validate the environment**
   Confirm you are targeting the correct workspace, account, and region.

7. Why the Resource Might Be Deleted

Resource block removed or renamed
The resource was deleted from the configuration or the name/identifier changed.

Configuration change
A change in a key attribute (e.g. instance type, subnet, or storage) may force replacement.

Using the wrong workspace or variables
You might be pointing to a different environment (e.g. prod instead of dev).

Terraform state mismatch
The state file may be outdated or corrupted.

Conditional logic or count/for_each change
A change in conditions may remove the resource from the configuration.

8. How to Prevent an Unsafe Deployment

- [x] Review the plan carefully before applying.
- [x] Use code review (pull requests) for all changes.
- [x] Enable plan approvals in CI/CD pipelines.
- [x] Use separate workspaces for dev, staging, and prod.
- [x] Use lifecycle rules (e.g. prevent_destroy = true) for critical resources.
- [x] Keep a remote state with versioning and locking.
- [x] Validate variables and environment settings.
- [x] Have a rollback or recovery plan if deletion happens by mistake.

9. Example Commands for Analysis

```
# See what will be deleted
terraform plan

# Inspect current state
terraform state show aws_instance.web

# Check recent changes
git log --oneline -5
git diff

# List dependencies (example)
terraform graph | dot -Tpng > graph.png
```

10. Practical Focus: Your Turn

1. Review a sample terraform plan output.
2. Identify the resource marked for deletion.
3. Explain the possible causes.
4. Write the steps you will take to investigate.
5. Describe how you will prevent an unsafe deployment.

11. Key Takeaways

- [x] Always understand what terraform plan is doing.
- [x] Investigate before you apply.
- [x] Know the impact of deleting a resource.
- [x] Use the right tools to verify state, code, and environment.
- [x] Prevent accidental deletion with safeguards.
- [x] Be ready to explain your reasoning clearly in an interview.

12. Pro Tip

Never run terraform apply without reviewing the plan. Use automated checks in CI/CD to block destructive changes in production environments.

The Live Technical Assessment

Learn how to approach an unfamiliar terminal task without panicking or blindly copying commands. Cover requirements, assumptions, safe changes, verification, and communication.

Practice challenge: Investigate a Linux service that will not start. Show your diagnostic sequence, explain the evidence, and verify recovery.

1. Understand the Task First

- Read the requirements carefully.
- Clarify what you are expected to achieve.
- Identify assumptions (OS, service name, expected behavior).
- Think out loud and communicate your approach.
- Avoid making random changes.

2. Initial Checks

Confirm the environment and service details.

```
# Check OS and hostname
uname -a
hostnamectl

# Check if the service exists
systemctl list-units --type=service
| grep -i <service-name>

# Check current status
systemctl status <service-name>
```

3. Investigate the Failure

Collect evidence to find the cause.

```
# View detailed service status
systemctl status <service-name> -l

# Check recent logs
journalctl -u <service-name> -n 50

# Look for errors in logs
journalctl -u <service-name> -p err

# Check configuration file
cat /etc/<service-name>/<config-file>

# Validate configuration (if available)
<service-name> --test-config
```

4. Common Causes to Check

- **Configuration errors** (e.g. wrong file path, invalid syntax)
- **Missing files or wrong permissions** (e.g. config, SSL certificates)
- **Network or port issues** (e.g. port already in use, firewall)
- **Dependency issues** (e.g. database not running, missing packages)
- **Resource limits** (e.g. low memory, disk full, too many processes)
- **Service user issues** (e.g. wrong user, permission denied)

5. Diagnose Step by Step (Example)

1. **Check the service status**
   What state is it in and what error do you see?
2. **Review the logs**
   Look for clear error messages or patterns.
3. **Check dependencies**
   Are required services, ports, or files available?
4. **Validate the configuration**
   Fix any syntax or reference errors.
5. **Check permissions**
   Ensure the service user can read required files.
6. **Make a safe fix**
   Apply the minimal change needed.
7. **Start the service again**
   `systemctl start <service-name>`
8. **Verify it is running**
   `systemctl status <service-name>`

6. Verification and Evidence

Confirm the issue is resolved and document your evidence.

```
# Check service status
systemctl status <service-name>

# Check logs for successful startup
journalctl -u <service-name> -n 20

# Verify the service is listening
ss -tulnp | grep <service-name>

# Test the application (if applicable)
curl -v http://localhost:<port>

# Check no new errors
journalctl -u <service-name> -n 20
```

7. Communication During the Assessment

- [x] Explain what you are doing and why.
- [x] Share your findings as you go.
- [x] Mention alternative possible causes.
- [x] Ask clarifying questions if requirements are unclear.
- [x] Be honest if you are unsure and explain how you will find out.
- [x] Summarize what you changed and why.

8. Example Output (Recovered Service)

Service is running and accessible.

```
$ systemctl status nginx
● nginx.service - A high performance web server
     Loaded: loaded (/usr/lib/systemd/system/nginx
     Active: active (running) since ...
$ ss -tulnp | grep :80
LISTEN   0      128    0.0.0.0:80    ... nginx
$ curl -I http://localhost
HTTP/1.1 200 OK
Server: nginx
...
```

9. What Interviewers Look For

- [x] A clear and structured approach.
- [x] Use of the right tools and commands.
- [x] Ability to interpret logs and system output.
- [x] Safe and minimal changes.
- [x] Verification that the issue is fully resolved.
- [x] Good communication and problem-solving thinking.

10. Practical Focus: Your Turn

1. Investigate a Linux service that will not start.
2. Identify the root cause and explain the evidence.
3. Apply a fix using safe changes.
4. Verify the service is running and working.
5. Write a short summary of what you did and why.

11. Key Takeaways

- [x] Don't guess. Investigate systematically.
- [x] Use logs and system tools to find the cause.
- [x] Make minimal and safe changes.
- [x] Verify the fix with real evidence.
- [x] Communicate clearly throughout the process.
- [x] This is real-world problem solving, not memorizing commands.

12. Pro Tip

If you don't know a command, use --help or man.

```
systemctl --help
man systemctl
```

Explain a Production Incident

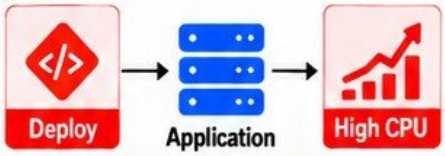
Learn to describe technical failures using symptoms, scope, timeline, evidence, mitigation, root cause, and prevention. Understand how to discuss a simulated incident honestly if you have no production experience.

Interview scenario: CPU usage rises after a deployment. Explain what you would inspect before concluding that the deployment caused the problem.

1. How to Structure Your Answer

1. **Symptoms**
   What happened? (e.g. high CPU)
2. **Scope**
   What was afffected? (one service, entire system, users, etc.)
3. **Timeline**
   When did it start? What changed before it started?
4. **Evidence**
   What data or logs confirm the issue?
5. **Mitigation**
   What did you do to reduce impact?
6. **Root Cause**
   What most likely caused it?
7. **Prevention**
   What would you do to avoid it in the future?

2. The Scenario: CPU Usage Rises



What You Know:

- A new version was deployed.
- Shortly after deployment, CPU usage increased on one or more servers/containers.
- Users are experiencing slow response times.
- You need to investigate before assuming the deployment caused the issue.

3. What to Inspect Before Blaming

1. Check overall system metrics (CPU, memory, load).
2. Identify which processes are using CPU.
3. Check if the new version is running.
4. Compare metrics before and after deployment.
5. Look for recent changes (deployments, configs, cron jobs).
6. Check application logs for errors or unusual activity.
7. Check infrastructure and dependency health (database, cache, external APIs).
8. Consider other possible causes (scheduled jobs, traffic spike, memory leaks, etc.).

4. Useful Commands for Investigation

Check CPU usage and top processes

```
top
htop
ps aux --sort=-%cpu | head
pidstat -u 1 5
```

Check system load

```
uptime
cat /proc/loadavg
```

Check running services

```
systemctl status <service-name>
```

Check application logs

```
journalctl -u <service-name> -n 100
tail -f /var/log/<app>/app.log
```

5. Example Investigation Flow

1. **Check current CPU and load**
   `top, htop, uptime`
2. **Identify the process using CPU**
   `ps aux --sort=-%cpui | head`
3. **Check if it's the new version**
   `systemctl status <service>`
   `docker ps | kubectl get pods`
4. **Review recent changes**
   deployment history, config changes, scheduled jobs
5. **Check application and system logs**
   `journalctl, app logs`
6. **Check dependencies**
   database, external APIs, network
7. **Compare metrics (before vs after)**
   monitoring tools (e.g. Prometheus, CloudWatch)

6. Sample Outputs (What to Look For)

High CPU process

```
PID   USER    %CPU  %MEM   COMMAND
1234  ubuntu  250.3  5.1   python app.py
5678  ubuntu  45.6   2.3   nginx
```

Service status

```
$ systemctl status myapp
● myapp.service - My Application
   Active: active (running)
   CPU: 250.3%
   Memory: 512M
```

Recent logs (example error)

```
$ journalctl -u myapp -n 5
ERROR   Database connection failed
WARN    High memory usage detected
ERROR   Request timeout
```

Kubernetes pod example

```
$ kubectl top pods
NAME          CPU(cores)    MEMORY(bytes)
myapp-7d6f9   1200m         512Mi
```

7. Possible Causes (Don't Assume)

- Application bug or memory/CPU leak
- Increased traffic or load
- Scheduled job or cron task
- Inefficient query or database issue
- Misconfiguration (e.g. logging too much)
- External dependency issues

8. How to Mitigate While Investigating

- ☑ Restart the affected service (if safe).
- ☑ Scale up resources temporarily.
- ☑ Disable non-critical features.
- ☑ Roll back the deployment (if evidence shows it's the cause).
- ☑ Communicate status and impact clearly to stakeholders.
- ☑ Continue investigating to confirm the root cause.

9. Key Takeaways

- ☑ Use evidence, not assumptions.
- ☑ Check multiple possible causes.
- ☑ Explain your investigation steps clearly in an interview.
- ☑ It's okay to say "I would check X, Y, Z" if you don't have real production experience.
- ☑ Show logical thinking and problem-solving, not memorized commands.

10. Practical Focus: Your Turn

1. Investigate a system with high CPU using the steps above.
2. Identify the most likely cause.
3. Explain the evidence for your conclusion.
4. Describe how you would mitigate the issue.
5. Write a short incident summary as you would in an interview.

VERIQTA

Behavioral Questions Engineers Actually Face

Prepare for questions about mistakes, disagreements, missed deadlines, learning unfamiliar technologies, and communicating during incidents.

Practical focus: Use the STAR method, Situation, Task, Action, Result, to explain a real experience. Distinguish personal contributions from team outcomes.

1. Common Behavioral Questions

- **Tell me about a mistake you made.** What did you learn?
- **Describe a time you disagreed with a team member.** How did you handle it?
- **Tell me about a time you missed a deadline.** What happened and what did you do?
- **Describe a time you had to learn an unfamiliar technology.** How did you approach it?
- **Tell me about a time you communicated during an incident.** What was your role?
- **Give an example of a project you improved.** What was the impact?
- **Describe a time you took initiative without being asked.**

2. Use the STAR Method

S — Situation
Set the context.
What was the situation?
Keep it clear and brief.

T — Task
What were you responsible for? What needed to be done?

A — Action
What specific actions did you take?
Focus on what you did, not just the team.

R — Result
What was the outcome?
What did you learn? Be honest about what you would do differently.

3. Example Answer (Realistic)

Question: Tell me about a time you had to learn an unfamiliar technology.

S — Situation
Our team needed to deploy an application using Kubernetes, which I had not used before.

T — Task
I was responsible for setting up the basic deployment and making sure the application could run in a test environment.

A — Action
I studied the official documentation, followed hands-on labs, and built a small prototype. I asked questions, tested in a non-production environment, and documented the steps for the team.

R — Result
We successfully deployed the application in the test environment, and I created a guide that helped other team members. This experience taught me to break down new technologies into small, practical steps.

4. How to Show Your Personal Contribution

- ✓ Use "I" statements to explain what you did.
- ✓ Be specific about your actions and decisions.
- ✓ Mention how you collaborated with others.
- ✓ If it was a team effort, clearly state your role.
- ✓ Avoid taking full credit for team outcomes.

5. Handling Challenging Topics

- **Mistakes** — Be honest, explain what happened, what you learned, and what you do differently now.
- **Disagreements** — Focus on respectful communication and finding a solution.
- **Missed Deadlines** — Explain the reason, what you did to recover, and how you prevent it now.
- **Learning New Technology** — Show your learning process and how you applied it.
- **Incident Communication** — Explain how you stayed calm, shared clear updates, and worked with the team.

6. Weak vs Strong Answers

Weak Answer	Strong Answer
✗ Vague and general ("I worked on it ...").	✓ Clear and specific details.
✗ No clear structure or outcome.	✓ Follows the STAR structure.
✗ Focuses only on the team, not you.	✓ Clearly explains your personal role.
✗ Avoids responsibility or gives excuses.	✓ Takes responsibility and shows ownership.
✗ No evidence of learning or improvement.	✓ Shows what you learned and how you improved.

7. Example Behavioral Question Matrix

Scenario	What Interviewers Look For
Mistake	Honesty, accountability, learning.
Disagreement	Communication, professionalism, problem-solving.
Missed deadline	Planning, recovery actions, prevention.
Learning new tech	Initiative, learning process, practical application.
Incident communication	Clarity, calmness, teamwork.
Improvement project	Initiative, impact, measurable results.

Key Takeaways

- ✓ Use the STAR method.
- ✓ Be honest and specific.
- ✓ Focus on your contribution.
- ✓ Show problem-solving and learning.
- ✓ Keep the answer relevant to the question.
- ✓ Avoid overly long or dramatic stories.
- ✓ Practice with real or simulated examples.
- ✓ Show growth and self-awareness.

9. Practical Focus: Your Turn

1. Choose one real experience (or a simulated scenario).
2. Write your answer using STAR.
3. Make sure you clearly explain your personal contribution.
4. Practice saying it out loud.
5. Prepare for follow-up questions (e.g. "What would you do differently?").

10. Pro Tip

Keep your answers concise and relevant. Aim for 1–2 minutes per question, then be ready to discuss more details if asked.

AI, Honesty, and the Skills You Actually Own

Understand the risks of AI-written resumes, invented project outcomes, copied GitHub repositories, and interview answers you cannot explain.

Practical focus: Audit five resume claims. Identify which are supported by evidence, which need rewriting, and which should be removed. Practice explaining work without AI assistance.

1. Where AI Can Be Helpful (and Safe)

- **Learning concepts**
  Get explanations and examples.
- **Improving wording**
  Make your writing clearer.
- **Practicing interview questions**
  Prepare and get feedback.
- **Generating ideas**
  Brainstorm project ideas or study plans.

2. Real Risks You Must Avoid

- **AI-written resumes**
  May include skills or experience you do not actually have.
- **Invented project outcomes**
  Claiming results you cannot prove.
- **Copied repositories**
  Using someone else's work and presenting it as yours.
- **Interview answers you cannot explain**
  Memorizing AI answers can be easily exposed.
- **Loss of credibility**
  Can damage your reputation and future opportunities.

3. Examples of Red Flags

- "Built a production-grade platform for millions of users" (but no evidence or explanation)
- "5+ years of experience with Kubernetes" (but cannot explain basic concepts)
- "Implemented CI/CD pipelines" (but copied a GitHub repository)
- "Expert in AWS, Terraform, and security" (plural skills listed without real use)
- Using AI to answer interview questions live (cannot explain the answer in detail)

4. Audit Your Resume Claims (Practical Exercise)

1. List five claims from your current resume.
2. Check what evidence you have for each claim.
3. Decide: Keep, Rewrite, or Remove.
4. Rewrite weak claims to be truthful and specific.
5. Practice explaining each claim without AI.

5. Resume Claim Audit Template

#	Claim on Resume	Evidence?	Action
1	Deployed a web application on AWS	✓	Keep
2	Implemented CI/CD with GitHub Actions	!	Rewrite
3	5+ years of Kubernetes experience	✗	Remove
4	Built a monitoring solution with Prometheus	✓	Keep
5	Led a team of 10 engineers	✗	Remove

6. How to Rewrite a Claim

Vague or Exaggerated (Don't use this)

"Built a scalable microservices platform used by thousands of users."

↓

Clear and Truthful (Use this)

"Built and deployed a simple microservices application using Docker and Kubernetes as a personal project. Practiced scaling, monitoring, and rolling updates in a lab environment."

7. Practice Explaining Without AI

- Choose a claim from your resume.
- Explain it out loud in simple terms.
- Be ready to answer follow-up questions such as "Why did you choose this tool?" or "What challenges did you face?"
- If you cannot explain it clearly, revise the claim or remove it.
- Repeat until you can explain it confidently without AI assistance.

8. Key Takeaways

- Use AI as a learning and support tool, not a replacement for your own work.
- **Be honest** about your skills and experience.
- Only include claims you can explain and prove.
- Rewrite vague claims to be specific and truthful.
- Remove claims that are not accurate.
- Practice explaining your work in your own words.
- Authenticity builds trust and long-term opportunities.

9. Real Interviewer Mindset

- **Interviewers** can tell when an answer is memorized or AI-generated.
- They will ask follow-up questions to test your understanding.
- They value honesty, curiosity, and willingness to learn.
- It is better to be honest and show your learning process than to pretend expertise you do not have.

10. Pro Tip

If you used AI to help you learn or improve your writing, it is okay to say so. Be transparent about how you used it and what you personally did.

VERIQTA

The Final DevOps Interview Simulation

Complete a realistic junior DevOps interview covering resume claims, project architecture, Linux troubleshooting, CI/CD, container failures, security decisions, and communication.

Final assessment: Review your resume, defend one project, troubleshoot one failure, explain one engineering trade-off, and identify what you still need to learn.

1. Interview Structure (What to Expect)

1. **Introduction & Resume Discussion**
   Tell me about yourself and walk me through your resume.
2. **Project Deep Dive**
   Explain the architecture, tools, challenges, and your role.
3. **Linux Troubleshooting**
   Solve a real-world problem in a terminal environment.
4. **CI/CD and DevOps Tooling**
   Explain pipelines, builds, deployments, and failures.
5. **Containers and Kubernetes**
   Diagnose issues and explain decisions.
6. **Security and Best Practices**
   Discuss IAM, networking, secrets, and risk.
7. **Behavioral and Communication**
   Explain your thinking, decisions, and teamwork.

2. Defend One Project (Example)

Be ready to explain:

- **The problem you solved**
  What was the goal and why it mattered?
- **The architecture**
  Tools, services, and how they work together.
- **Your specific contributions**
  What you built, configured, or improved.
- **Challenges and how you solved them**
  What went wrong and what you did about it.
- **The results**
  What changed or improved because of your work?
- **What you learned**
  What you would do differently next time?

3. Troubleshoot One Failure (Example)

Scenario: A web application is not reachable after a deployment.

1. Check the service status
   ```
   systemctl status <service-name>
   ```
2. Check logs for errors
   ```
   journalctl -u <service-name> -n 50
   ```
3. Verify the application is listening
   ```
   ss -tulpn | grep <port>
   ```
4. Test local and remote connectivity
   ```
   curl -v http://localhost:<port>
   curl -v http://<server-ip>:<port>
   ```
5. Check recent changes
   ```
   git log --oneline -5
   kubectl get pods -o wide
   ```
6. **Identify the cause and fix safely**
   Explain what you found, what you changed, and how you verified the fix.

4. Answer Engineering Trade-Off Questions

Be ready to discuss trade-offs such as:

- **Cost vs. reliability**
  E.g. managed services vs. self-managed.
- **Security vs. convenience**
  E.g. strict access controls vs. fast delivery.
- **Speed vs. stability**
  E.g. frequent deployments vs. longer testing.
- **Simplicity vs. flexibility**
  E.g. simple architecture vs. complex requirements.
- **Build vs. buy**
  E.g. open source tools vs. commercial services.

5. Security Decision Scenario

Example question:

"You need to store database credentials for a containerized application. What options would you consider and what are the trade-offs?"

- **Use environment variables** (understand the risks).
- **Use a secrets manager** (e.g. AWS Secrets Manager).
- **Use Kubernetes secrets** (with proper access control).
- Restrict access with IAM/RBAC.
- Explain your reasoning and choose the safest approach.

6. Container/Kubernetes Failure Example

Scenario: A Pod is in CrashLoopBackOff.

```
$ kubectl get pods
NAME                READY    STATUS
web-app-7d6f9       0/1      CrashLoopBackOff
$ kubectl describe pod web-app-7d6f9
# Look at Events, Exit Code, and Message
$ kubectl logs web-app-7d6f9 --previous
# Check the previous container logs
$ kubectl get events --sort-by=.lastTimestamp
# Look for image pull, probe or resource issues
```

1. Explain what CrashLoopBackOff means.
2. Identify what evidence you would inspect.
3. Describe possible causes.
4. Explain how you would fix and verify the resolution.

7. Final Assessment Checklist

- [x] Review your resume and be ready to explain every claim.
- [x] Defend one of your projects in detail.
- [x] Troubleshoot one failure step by step.
- [x] Explain one engineering trade-off.
- [x] Answer at least one behavioral question using STAR.
- [x] Identify what you still need to learn and how you plan to learn it.

8. Key Success Tips

- Be honest about your experience.
- Show your thinking process.
- Communicate clearly and step by step.
- It's okay to say "I don't know" and explain how you would find the answer.
- Focus on what you can explain and what you actually did.
- Stay calm, be structured, and show a willingness to learn.

9. Final Reflection (Your Turn)

Write a short reflection:

1. What went well in the simulation?
2. What was challenging?
3. What do you still need to learn?
4. What is your plan to improve?

10. Pro Tip

Practice out loud. Time yourself. The more you practice, the more natural and confident you will sound in a real interview.

Follow @veriqta @veriqta | @veriqta | @veriqta | @veriqta | @veriqta

VERIQTA